Universidad Nacional de Tucumán

Facultad de Ciencias Exactas y Tecnología



Integración Numérica y Cálculo de Áreas Irregulares

Por:
RIVERA OCTTAVIANO, Ramiro Tomas - Ingeniería en Informática
NIEVA OVEJERO, Joel Esteban - Licenciatura en Informática
NOTA, José Ignacio - Licenciatura en Informática

2026
Departamento Ciencias de Computación

Índice:

1. Resumen....3
2. Objetivos....3
3. Fundamentos Teóricos....4
3.1. El Método de Monte Carlo.... 4
3.2. Integración numérica: el problema de las integrales sin primitiva.... 4
3.3. Método del Valor Medio.... 5
3.4. Método de Acierto y Error (Muestreo Aleatorio).... 5
4. Decisiones de Diseño e Implementación en MPI....5
4.1. Métricas de rendimiento paralelo....6
4.2. Generación Independiente de Semillas Aleatorias....6
4.3. Distribución de Carga Cíclica....6
4.4. Funciones de MPI.... 7
5. Entornos de Pruebas y Metodología de Medición....7
Entorno 1: Hardware Local (Multihilo / SMT).... 8
Entorno 2: Clúster de Alto Rendimiento (HPC)....8
6. Resultados....8
6.1. Aproximación de las integrales con distinta cantidad de procesos....8
6.2. Rendimiento en Entorno Local....12
6.3. Rendimiento en el Clúster HPC (Escalabilidad Pura).... 17
Clúster: 48 GB de RAM y 12 Cores (Valor Medio).... 17
Clúster: 96 GB de RAM y 24 Cores (Valor Medio).... 19
Clúster: 192 GB de RAM y 48 Cores (Valor Medio).... 21
Clúster: 384 GB de RAM y 96 Cores (Valor Medio).... 24
6.4. Comparativa de Rendimiento entre Métodos.... 28
1. Gráfica Comparativa: Entorno Local (N=10^8).... 30
2. Gráfica Comparativa: Clúster HPC (N=10^10).... 31
7. Conclusiones.... 31

1. Resumen

El presente trabajo aborda la resolución numérica de integrales definidas para las funciones e^{-x^2} y $\sin(x^2)$ mediante la aplicación del Método de Monte Carlo, evaluando y contrastando dos de sus variantes: el Método del Valor Medio y el Método de Acierto y Error (Muestreo Aleatorio). Dado que estos enfoques probabilísticos requieren un volumen masivo de iteraciones para converger con precisión, se desarrolló una implementación en paralelo utilizando el lenguaje C y el estándar MPI (Message Passing Interface).

El desarrollo incluye una justificación profunda de las decisiones arquitectónicas del código, tales como la gestión independiente de semillas pseudoaleatorias, la eliminación de cuellos de botella por sincronización y el uso de fases de "calentamiento" para evitar distorsiones en la medición. Finalmente, se presenta un análisis exhaustivo de rendimiento evaluando el algoritmo bajo cargas de hasta 10^{10} simulaciones, contrastando su comportamiento en un entorno de hardware local (multihilo) frente a un Clúster de Alto Rendimiento. Los resultados demuestran cómo un diseño paralelo pulido permite obtener escalabilidad, un *SpeedUp* casi lineal y una altísima eficiencia computacional.

2. Objetivos

- Comprender cómo se usa el método de Monte Carlo para resolver una integral definida en una dimensión y para estimar el área de una figura irregular.
- Paralelizar el algoritmo de cálculo distribuyendo la carga de manera cíclica entre múltiples procesos para reducir los tiempos de ejecución.
- Comparar el método del valor medio con el muestreo aleatorio de acierto y fallo, y analizar la convergencia al aumentar el número de simulaciones..
- Medir el tiempo de ejecución con distintas cantidades de procesos y de simulaciones, y calcular el speedup y la eficiencia usando la mediana de múltiples ejecuciones.

3. Fundamentos Teóricos

3.1. El Método de Monte Carlo

Es una técnica de simulación estadística que usa números aleatorios para calcular cantidades que no se pueden obtener fácilmente de forma analítica. Se originó en la década de 1940 en el Laboratorio de Los Álamos, en el contexto del Proyecto Manhattan, donde se necesitaba simular el comportamiento de neutrones, un problema muy difícil de abordar con métodos deterministas. Su desarrollo se atribuye a Stanislaw Ulam y John von Neumann, y el nombre, propuesto por Nicholas Metropolis, hace referencia al casino de Monte Carlo, por su naturaleza aleatoria.

Las principales características del método son:

- Usa números aleatorios para simular eventos con probabilidad conocida.
- Permite aproximar soluciones de problemas difíciles o imposibles de resolver de forma analítica.
- Su precisión mejora con el número de simulaciones, aunque lentamente.
- Es especialmente útil en problemas con muchas variables (dimensiones altas).

¿Cómo funciona?

El principio básico es repetir muchas veces un experimento aleatorio y promediar los resultados. Dos resultados de la probabilidad lo justifican:

- **Ley de los grandes números:** el promedio de N valores independientes de una variable aleatoria se acerca a su valor esperado cuando N crece.
- **Teorema central del límite:** el error de ese promedio se distribuye aproximadamente como una campana de Gauss con desviación $\sigma/\sqrt{N}$, donde σ es la desviación estándar de la variable.

Por ejemplo, si se lanza un dado de seis caras pocas veces, la frecuencia de cada número puede estar lejos de 1/6; si se lo lanza muchas veces, se acerca a 1/6. Monte Carlo usa una computadora para hacer esos lanzamientos a gran escala.

3.2. Integración numérica: el problema de las integrales sin primitiva

La integral definida de una función positiva es el área bajo su curva. Cuando se conoce una primitiva, se calcula con la regla de Barrow, pero muchas funciones no la tienen en forma elemental. En este trabajo se usan dos de ellas en el intervalo [0, 2]:

exp(−x²), y sin(x²), cuya integral es una integral de Fresnel. En estos casos se recurre a la integración numérica, con métodos deterministas (trapecio, Simpson) o con métodos de Monte Carlo.

Algo parecido ocurre con el área de una figura irregular: si su contorno no tiene una ecuación simple, no hay una fórmula para su área, pero sí se puede decidir, para cada punto del plano, si está dentro o fuera de la figura. Eso alcanza para aplicar Monte Carlo.

3.3. Método del Valor Medio

Este enfoque se basa en el Teorema del Valor Medio para integrales. Consiste en evaluar la función matemática en N puntos generados aleatoriamente de manera uniforme dentro del intervalo de integración $[a, b]$. Al promediar los resultados de estas evaluaciones, se obtiene la "altura promedio" de la función en dicho intervalo. Finalmente, el área (la integral) se calcula multiplicando esta altura promedio por el ancho del intervalo $(b - a)$. Es un método altamente eficiente computacionalmente, ya que requiere generar un solo número aleatorio x por iteración.

3.4. Método de Acierto y Error (Muestreo Aleatorio)

A diferencia del Valor Medio, este método interpreta la integral como un problema geométrico de cálculo de áreas. Se define una "caja" rectangular (Bounding Box) que encierra completamente la curva de la función en el intervalo deseado. Posteriormente, se disparan N puntos aleatorios con coordenadas (x, y) dentro de esta caja.

Si el punto cae por debajo de la curva (y por encima del eje X para áreas positivas, o por debajo del eje X para áreas negativas como ocurre con el $\sin(x^2)$), se contabiliza como un "acierto". La aproximación de la integral se obtiene multiplicando el área total de la caja geométrica por la proporción de aciertos sobre el total de puntos generados. Al requerir dos números aleatorios por iteración y comprobaciones condicionales, este método implica una mayor carga computacional.

4. Decisiones de Diseño e Implementación en MPI

La programación paralela divide un trabajo entre varias unidades de cálculo que funcionan al mismo tiempo. MPI (Message Passing Interface) es una especificación

estándar para programar en memoria distribuida: se lanzan P copias del mismo programa, cada una es un proceso del sistema operativo con su propia memoria, y los procesos solo pueden intercambiar datos mediante mensajes explícitos. Cada proceso conoce su rank (de 0 a P−1) y la cantidad total P, y con eso decide qué parte del trabajo le toca. El estándar surgió del trabajo del MPI Forum, iniciado en 1992, que publicó MPI-1 en 1994; hay varias implementaciones, como Open MPI, MPICH y MS-MPI

4.1. Métricas de rendimiento paralelo

S(P) = T(1) / T(P)

E(P) = S(P) / P

- **Speedup S(P):** cuántas veces más rápido es el programa con P procesos que con uno. Lo ideal es S = P.
- **Eficiencia E(P):** qué fracción del ideal se logra; E = 1 es escala perfecta.
- **Mediana:** el tiempo de cada combinación se resume con la mediana de 30 corridas, porque una corrida atípica (por ejemplo, una interferencia del sistema) distorsiona el promedio pero no la mediana.

4.2. Generación Independiente de Semillas Aleatorias

Una de las decisiones arquitectónicas más importantes fue el correcto manejo del estado de los generadores de números pseudoaleatorios (srand).

- **El Problema:** Si todos los procesos MPI inicializan su semilla basándose únicamente en el tiempo actual del sistema (time(NULL)), todos generarían exactamente la misma secuencia de puntos, anulando el propósito de la simulación. Además, inicializar la semilla *dentro* de las funciones de cálculo provocaba que iteraciones consecutivas muy rápidas repitieran el patrón.
- **La Solución Adoptada:** Se extrajo la inicialización de la semilla al hilo principal (main) para sembrar el estado una única vez por proceso. Para garantizar independencia estadística total, se calculó la semilla como: semilla_base + 7919 * rank. El uso de un número primo grande (7919) multiplicado por el identificador del proceso (rank) asegura que las secuencias de generación de cada núcleo estén completamente distanciadas y no se superpongan. Además, la semilla_base se difunde mediante MPI_Bcast para permitir pruebas reproducibles.

4.3. Distribución de Carga Cíclica

Para dividir el trabajo de los N puntos entre los P procesos disponibles, se optó por una distribución cíclica (implementada mediante el bucle for (long long i = rank; i < n; i += size)). Esta técnica fue elegida frente a la distribución por bloques debido a su

simplicidad geométrica y su capacidad intrínseca para balancear la carga de trabajo sin requerir cálculos adicionales para segmentar los intervalos.

4.4. Funciones de MPI

Las funciones de MPI que usa el programa son:

1. **MPI_Init:** inicializa el entorno. El lanzador (mpirun) crea P procesos y cada uno ejecuta el programa completo; MPI_Init prepara los canales de comunicación entre ellos.
2. **MPI_Comm_rank:** devuelve el identificador del proceso (rank) dentro del comunicador global MPI_COMM_WORLD. Determina qué iteraciones le tocan.
3. **MPI_Comm_size:** devuelve la cantidad total de procesos P.
4. **MPI_Bcast:** el proceso 0 envía la semilla base a todos, para que todos partan de la misma y las corridas sean reproducibles.
5. **MPI_Barrier:** ningún proceso continúa hasta que todos llegaron. Se usa antes y después de cada medición, para que todos empiecen a la vez y el tiempo medido sea el del proceso más lento.
6. **MPI_Wtime:** reloj de pared de alta resolución. Se mide en el proceso 0 entre las dos barreras.
7. **MPI_Reduce:** suma las sumas parciales de todos los procesos (operación MPI_SUM, tipo MPI_DOUBLE) y deja el total en el proceso 0.
8. **MPI_Finalize:** cierra el entorno y libera los recursos

5. Entornos de Pruebas y Metodología de Medición

Para evaluar el impacto de la arquitectura de hardware en la escalabilidad del algoritmo, las simulaciones se ejecutaron en dos entornos de procesamiento con características distintas. En ambos casos, se automatizó la toma de métricas mediante un *script* en Python que promedió múltiples ejecuciones (entre 5 y 50, dependiendo de la carga) para aislar el ruido del sistema operativo y obtener tiempos estadísticamente robustos.

Entorno 1: Hardware Local (Multihilo / SMT)

- **Características:** Subsistema de Windows para Linux (WSL2 - Ubuntu) ejecutándose sobre un procesador de escritorio con 8 núcleos físicos y 16 hilos lógicos (*Hyper-Threading*).
- **Propósito:** Analizar el comportamiento del algoritmo al saturar los núcleos físicos reales y evaluar el rendimiento marginal que aportan los hilos virtuales al ejecutar entre 12 y 16 procesos MPI.

Entorno 2: Clúster de Alto Rendimiento (HPC)

- **Características:** Nodos de procesamiento dedicados con configuraciones de múltiples núcleos físicos (evaluado en instancias de 12 y 24 cores) y alta capacidad de memoria RAM.
- **Propósito:** Medir la escalabilidad pura del código manteniendo una relación 1:1 entre procesos MPI y núcleos físicos reales (sin *oversubscribe* ni cambio de contexto), permitiendo escalar el tamaño del problema (N) a magnitudes masivas (10^{10} simulaciones).

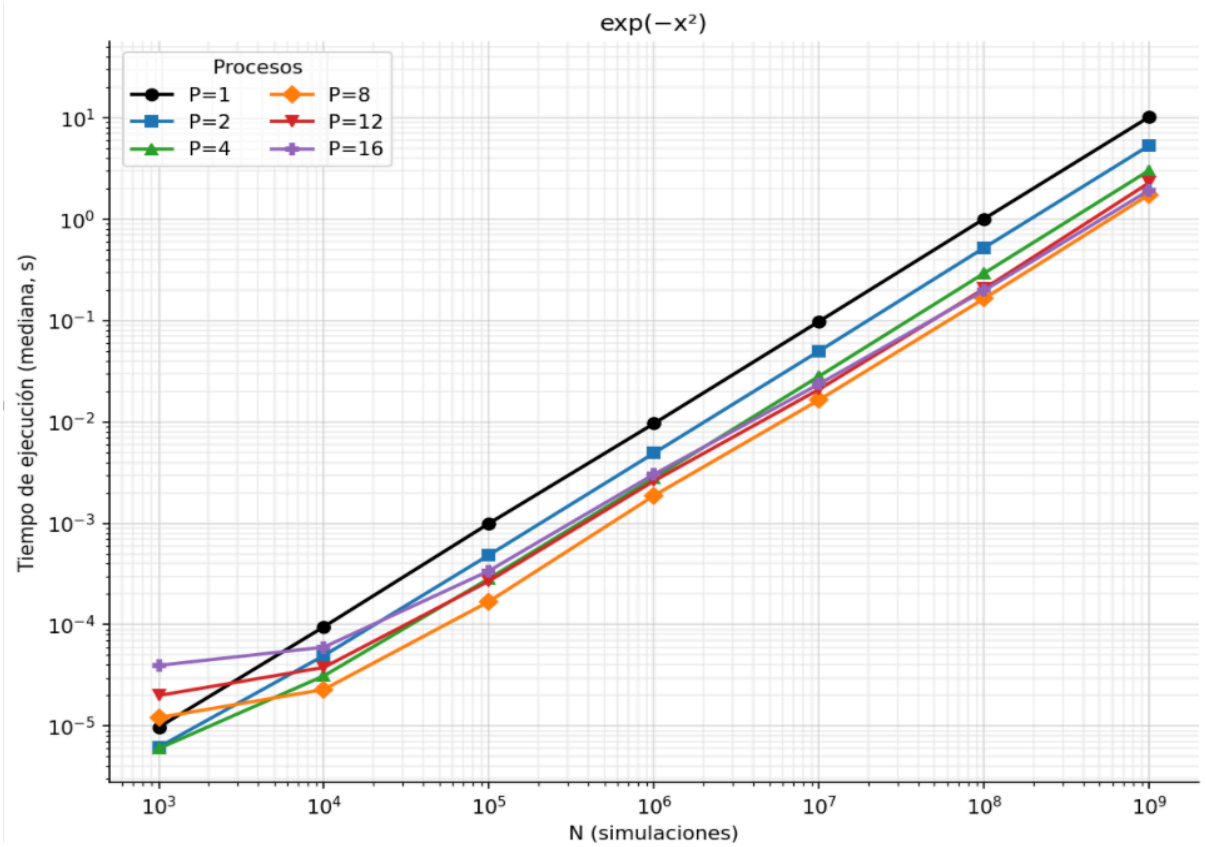
6. Resultados

6.1. Aproximación de las integrales con distinta cantidad de procesos

Para validar la precisión del algoritmo probabilístico y confirmar empíricamente la Ley de los Grandes Números, se realizaron mediciones escalando progresivamente el volumen de simulaciones (N) desde 10^3 hasta 10^9. Como es característico del Método de Monte Carlo, la exactitud matemática de la aproximación no depende de la cantidad de procesos empleados, sino del número total de muestras aleatorias evaluadas.

En los Gráficos 1, 2 y 3 que se presentan a continuación, se visualiza claramente este comportamiento. A medida que se inyecta una mayor cantidad de iteraciones al sistema, el valor calculado por el algoritmo converge de forma estable hacia el valor exacto de la integral para ambas funciones (e^{-x^2} y $\sin(x^2)$). En consecuencia, se observa una caída drástica y constante del error relativo. Esta representación visual sintetiza el comportamiento estocástico del método y demuestra su eficacia matemática antes de someterlo al análisis de estrés de hardware.

Gráfico 1: Tiempo de ejecución vs. cantidad de simulaciones para distinta cantidad de procesos:



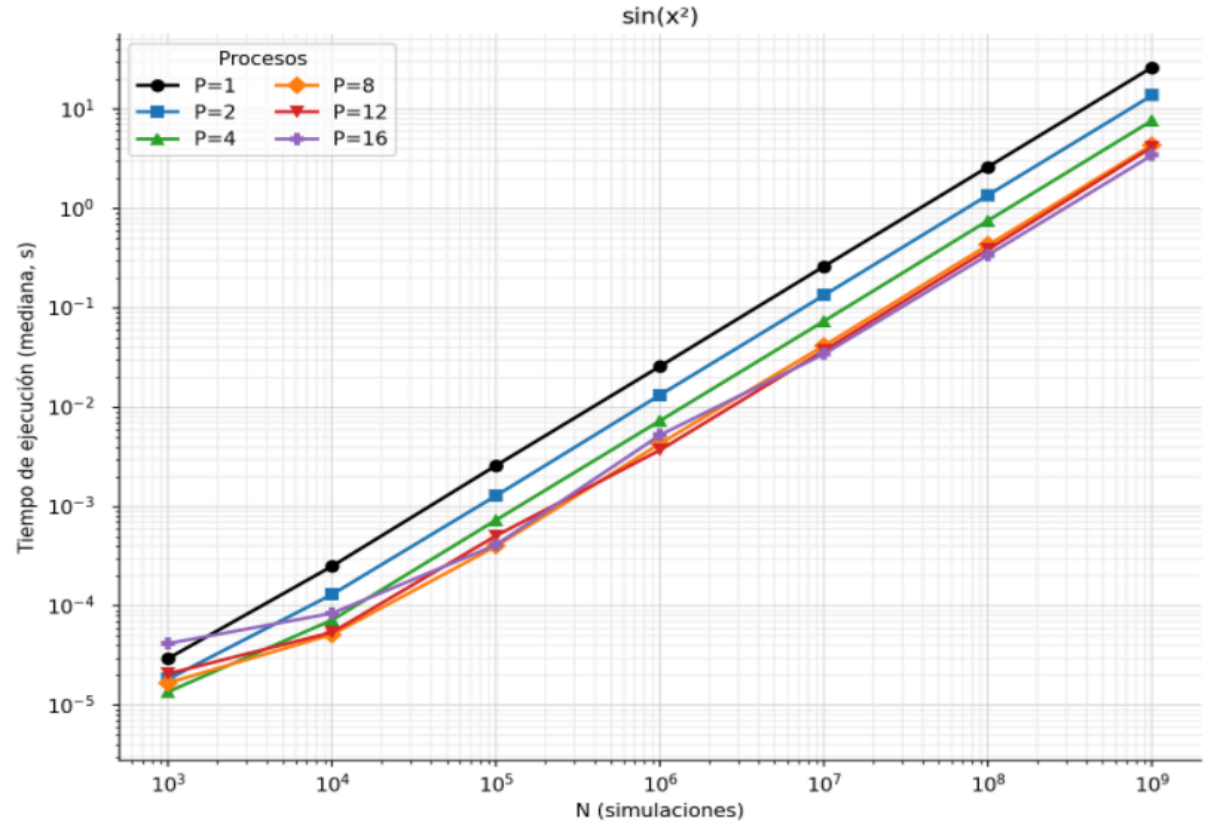
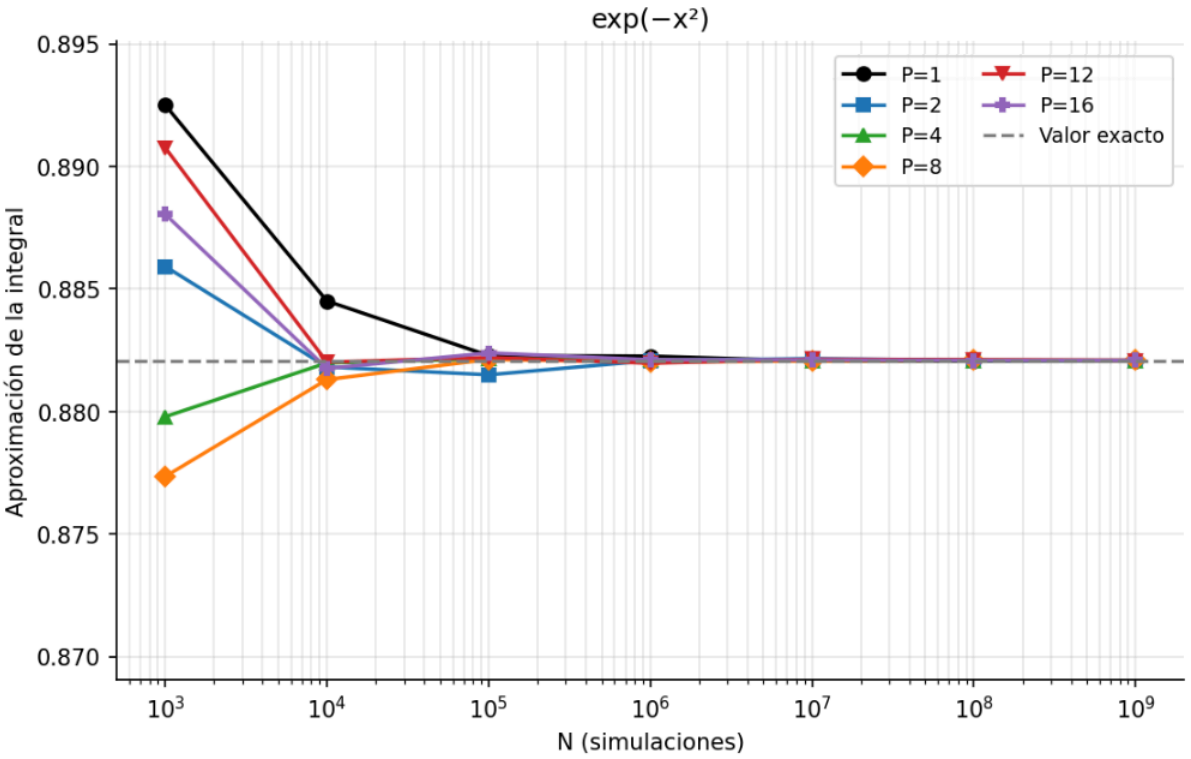


Gráfico 2: Aproximación vs. cantidad de simulaciones para distinta cantidad de procesos:



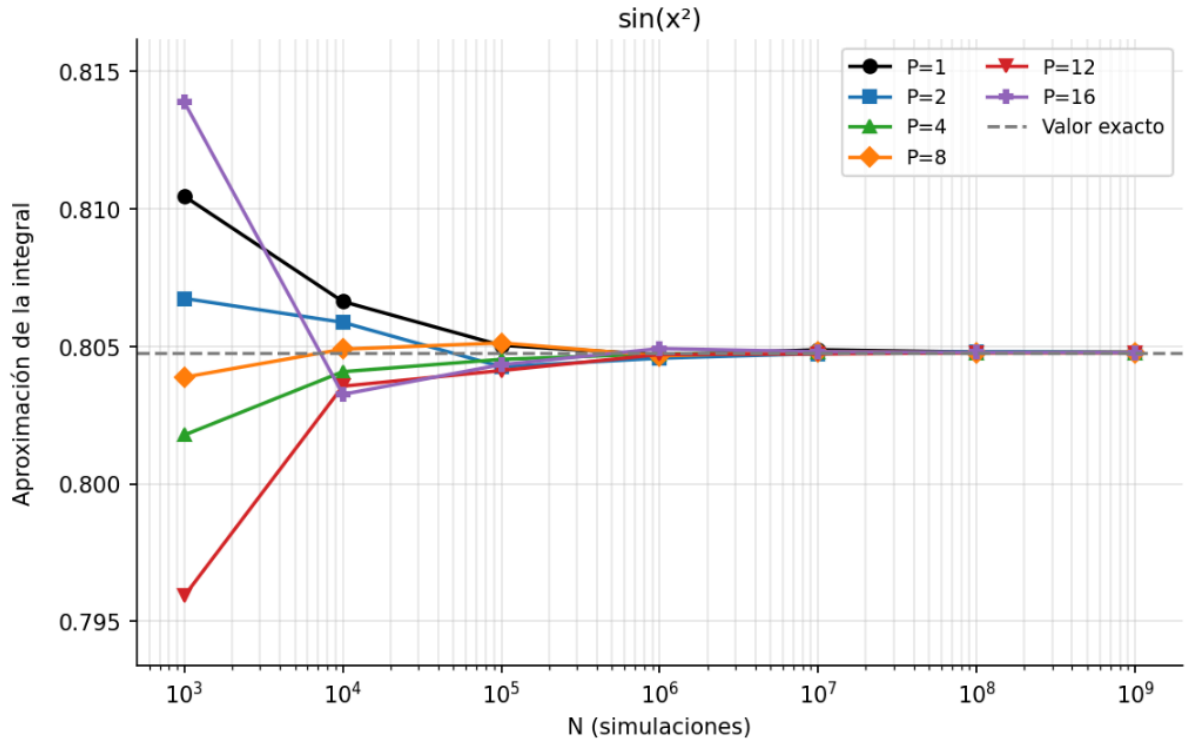
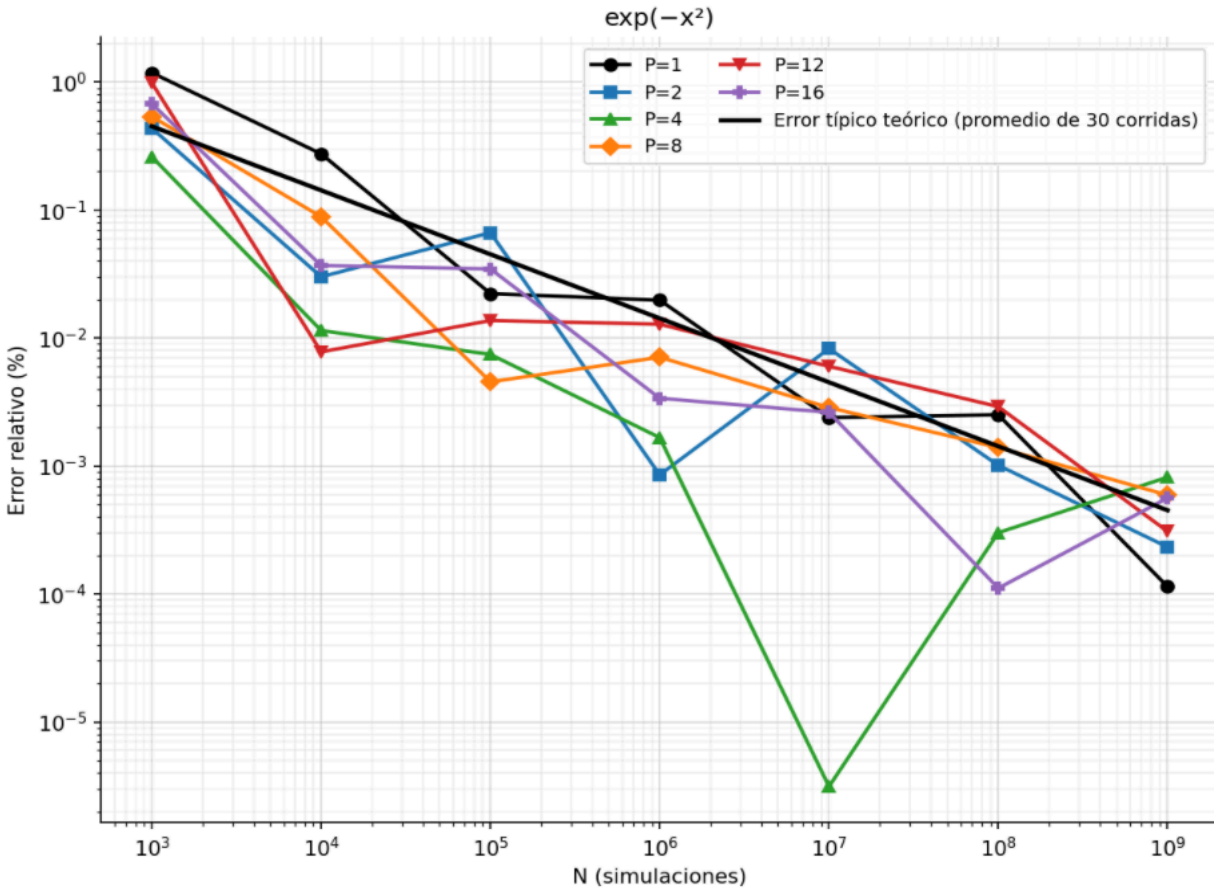
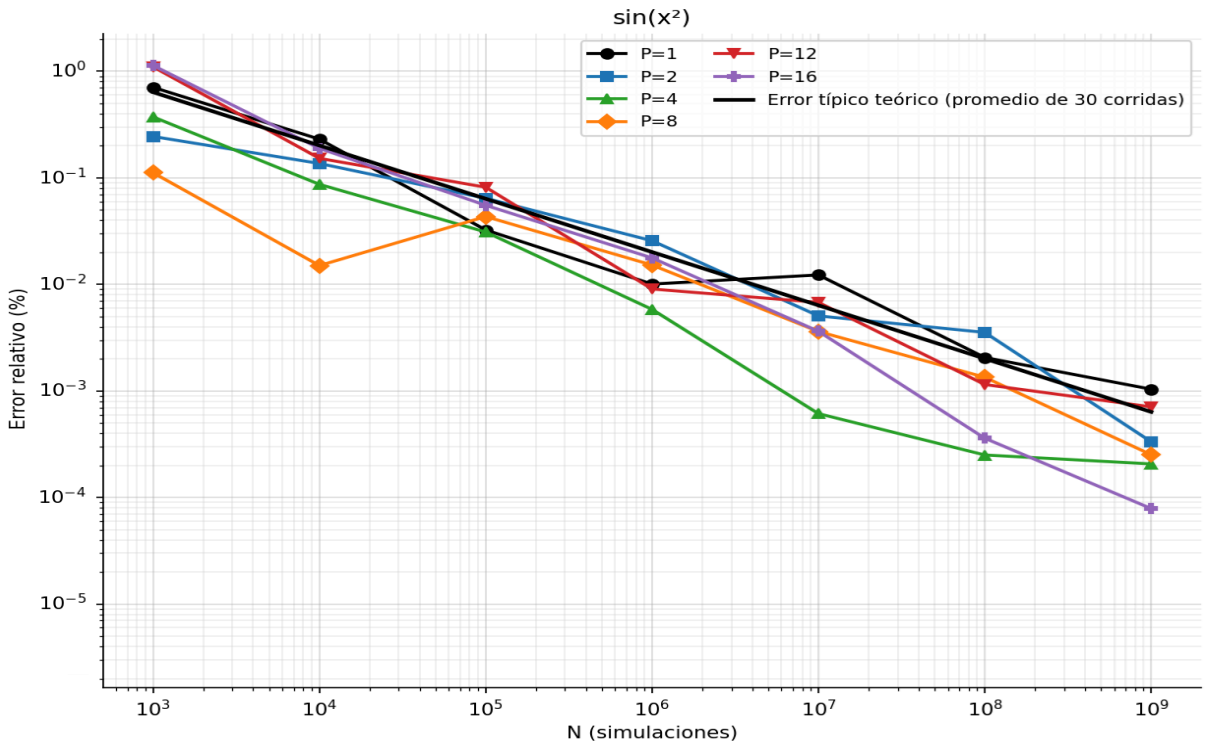


Gráfico 3: Error relativo vs. cantidad de simulaciones para distinta cantidad de procesos:





6.2. Rendimiento en Entorno Local

Tabla 1: cálculo del speedup para distinta cantidad de procesos usando N = 10^3 simulaciones:

Función	Procesos	Tiempo de ejecución (s)	Aproximación	Speedup	Eficiencia
$\exp(-x^2)$	1	0.000009753	0.8925045974	1.00	1.000
$\exp(-x^2)$	2	0.000006207	0.8859196125	1.57	0.786
$\exp(-x^2)$	4	0.000006006	0.8797842567	1.62	0.406
$\exp(-x^2)$	8	0.000012118	0.8773446849	0.80	0.101
$\exp(-x^2)$	12	0.000020038	0.8907623063	0.49	0.041
$\exp(-x^2)$	16	0.000039670	0.8880485210	0.25	0.015
$\text{sen}(x^2)$	1	0.000029791	0.8104589835	1.00	1.000
$\text{sen}(x^2)$	2	0.000018465	0.8067419758	1.61	0.807
$\text{sen}(x^2)$	4	0.000013750	0.8017755602	2.17	0.542
$\text{sen}(x^2)$	8	0.000016862	0.8038789027	1.77	0.221
$\text{sen}(x^2)$	12	0.000020865	0.7959246249	1.43	0.119
$\text{sen}(x^2)$	16	0.000042024	0.8138820359	0.71	0.044

Tabla 2: Misma tabla, ahora usando N = 10^4 simulaciones:

Función	Procesos	Tiempo de ejecución (s)	Aproximación	Speedup	Eficiencia
$\exp(-x^2)$	1	0.000095374	0.8845055374	1.00	1.000
$\exp(-x^2)$	2	0.000049603	0.8818158905	1.92	0.961
$\exp(-x^2)$	4	0.000031294	0.8819803699	3.05	0.762
$\exp(-x^2)$	8	0.000022953	0.8812998786	4.16	0.519
$\exp(-x^2)$	12	0.000037882	0.8820126441	2.52	0.210
$\exp(-x^2)$	16	0.000059963	0.8817559715	1.59	0.099
$\text{sen}(x^2)$	1	0.000252448	0.8066306579	1.00	1.000
$\text{sen}(x^2)$	2	0.000131291	0.8058733319	1.92	0.961
$\text{sen}(x^2)$	4	0.000072521	0.8040779922	3.48	0.870
$\text{sen}(x^2)$	8	0.000051908	0.8048973361	4.86	0.608
$\text{sen}(x^2)$	12	0.000054518	0.8035509712	4.63	0.386
$\text{sen}(x^2)$	16	0.000084660	0.8032566599	2.98	0.186

Tabla 3: Misma tabla, ahora usando N = 10^5 simulaciones:

Función	Procesos	Tiempo de ejecución (s)	Aproximación	Speedup	Eficiencia
exp(-x^2)	1	0.000996082	0.8822776460	1.00	1.000
exp(-x^2)	2	0.000486383	0.8814942361	2.05	1.024
exp(-x^2)	4	0.000284945	0.8821470923	3.50	0.874
exp(-x^2)	8	0.000169228	0.8821215349	5.89	0.736
exp(-x^2)	12	0.000269222	0.8822023368	3.70	0.308
exp(-x^2)	16	0.000339279	0.8823875984	2.94	0.183
sen(x^2)	1	0.002600022	0.8050372562	1.00	1.000
sen(x^2)	2	0.001292069	0.8042598901	2.01	1.006
sen(x^2)	4	0.000734754	0.8045270860	3.54	0.885
sen(x^2)	8	0.000400727	0.8051249638	6.49	0.811
sen(x^2)	12	0.000510431	0.8041204358	5.09	0.424
sen(x^2)	16	0.000413574	0.8043322804	6.29	0.393

Tabla 4: Misma tabla, ahora usando N = 10^6 simulaciones:

Función	Procesos	Tiempo de ejecución (s)	Aproximación	Speedup	Eficiencia
exp(-x^2)	1	0.009637404	0.8822557867	1.00	1.000
exp(-x^2)	2	0.004922515	0.8820738620	1.96	0.979
exp(-x^2)	4	0.002800879	0.8820961023	3.44	0.860
exp(-x^2)	8	0.001875609	0.8820189381	5.14	0.642
exp(-x^2)	12	0.002615716	0.8819682046	3.68	0.307
exp(-x^2)	16	0.003040742	0.8821112456	3.17	0.198
sen(x^2)	1	0.025849042	0.8046954880	1.00	1.000
sen(x^2)	2	0.013315491	0.8045699243	1.94	0.971
sen(x^2)	4	0.007331874	0.8047296160	3.53	0.881
sen(x^2)	8	0.004315265	0.8046543590	5.99	0.749
sen(x^2)	12	0.003713038	0.8047036645	6.96	0.580
sen(x^2)	16	0.005249833	0.8049189723	4.92	0.308

Tabla 5: Misma tabla, ahora usando N = 10^7 simulaciones:

Función	Procesos	Tiempo de ejecución (s)	Aproximación	Speedup	Eficiencia
exp(-x^2)	1	0.097894181	0.8820602583	1.00	1.000
exp(-x^2)	2	0.049889949	0.8821543748	1.96	0.981
exp(-x^2)	4	0.028194386	0.8820814184	3.47	0.868

exp(-x^2)	8	0.016466190	0.8820561324	5.95	0.743
exp(-x^2)	12	0.020929311	0.8821344442	4.68	0.390
exp(-x^2)	16	0.023625143	0.8821045589	4.14	0.259
sen(x^2)	1	0.261734990	0.8048751671	1.00	1.000
sen(x^2)	2	0.134048298	0.8047357639	1.95	0.976
sen(x^2)	4	0.073617389	0.8047814190	3.56	0.889
sen(x^2)	8	0.041627497	0.8048055665	6.29	0.786
sen(x^2)	12	0.037331025	0.8047219542	7.01	0.584
sen(x^2)	16	0.034364278	0.8048056937	7.62	0.476

Tabla 6: Misma tabla, ahora usando N = 10^8 simulaciones:

Función	Procesos	Tiempo de ejecución (s)	Aproximación	Speedup	Eficiencia
exp(-x^2)	1	1.006344502	0.8820592195	1.00	1.000
exp(-x^2)	2	0.521132770	0.8820903298	1.93	0.966
exp(-x^2)	4	0.292391606	0.8820787308	3.44	0.860
exp(-x^2)	8	0.164438727	0.8820936960	6.12	0.765
exp(-x^2)	12	0.205808811	0.8821070681	4.89	0.407
exp(-x^2)	16	0.197588444	0.8820823708	5.09	0.318
sen(x^2)	1	2.617132692	0.8047930717	1.00	1.000
sen(x^2)	2	1.361864202	0.8048049883	1.92	0.961
sen(x^2)	4	0.754887937	0.8047785066	3.47	0.867
sen(x^2)	8	0.427893153	0.8047656722	6.12	0.765
sen(x^2)	12	0.386281101	0.8047672450	6.78	0.565
sen(x^2)	16	0.340797818	0.8047793857	7.68	0.480

Tabla 7: Misma tabla, ahora usando N = 10^9 simulaciones:

Función	Procesos	Tiempo de ejecución (s)	Aproximación	Speedup	Eficiencia
exp(-x^2)	1	10.172298271	0.8820823985	1.00	1.000
exp(-x^2)	2	5.319886576	0.8820793366	1.91	0.956
exp(-x^2)	4	3.020970074	0.8820741735	3.37	0.842
exp(-x^2)	8	1.744935740	0.8820866187	5.83	0.729
exp(-x^2)	12	2.284145098	0.8820786701	4.45	0.371
exp(-x^2)	16	1.926481137	0.8820863581	5.28	0.330
sen(x^2)	1	26.230971862	0.8047848460	1.00	1.000
sen(x^2)	2	13.703932071	0.8047738026	1.91	0.957
sen(x^2)	4	7.645330568	0.8047781525	3.43	0.858
sen(x^2)	8	4.346741741	0.8047785195	6.03	0.754

sen(x^2)	12	4.133195466	0.8047707839	6.35	0.529
sen(x^2)	16	3.429483826	0.8047771239	7.65	0.478

Gráfico 4: Tiempo de ejecución y speedup vs. cantidad de procesos para $\exp(-x^2)$, para $1x10^7$ y $1x10^9$:

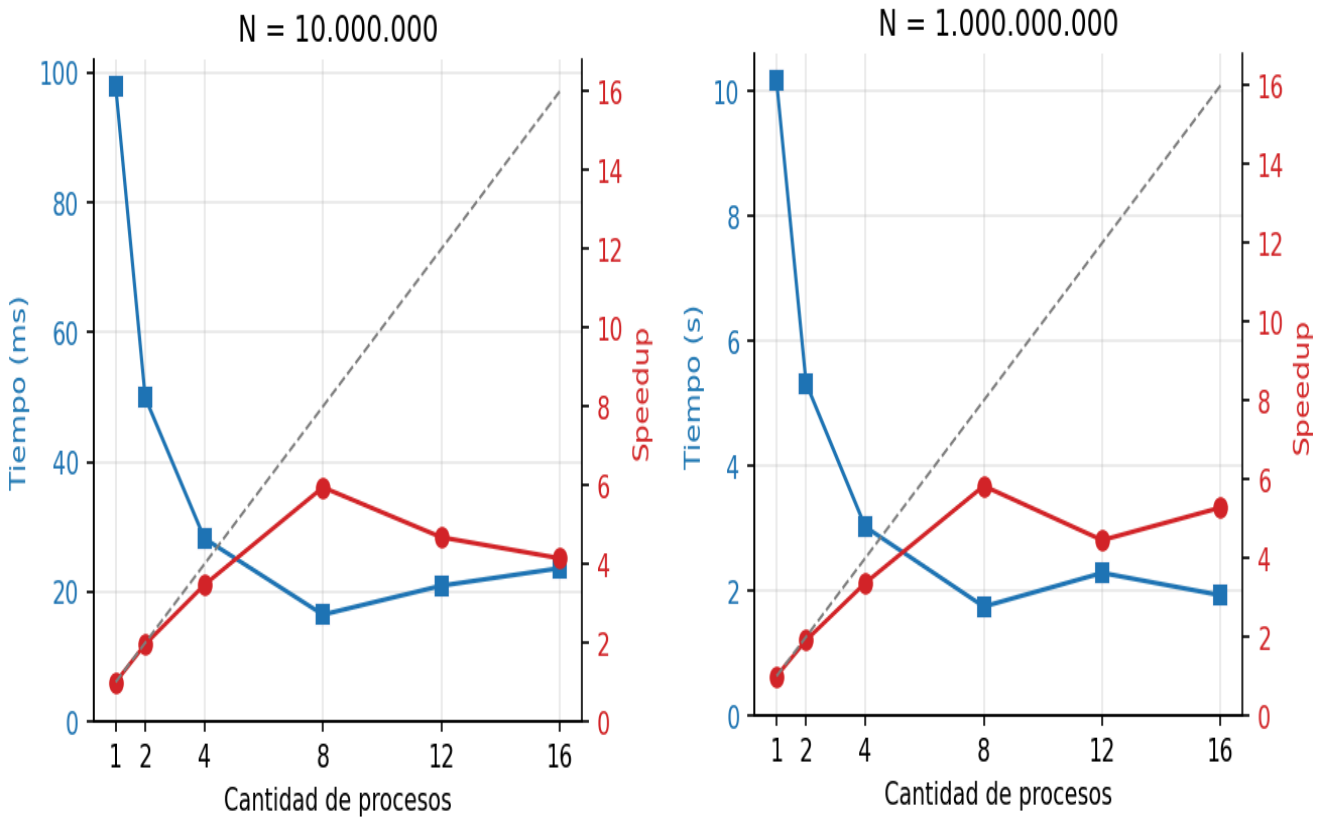
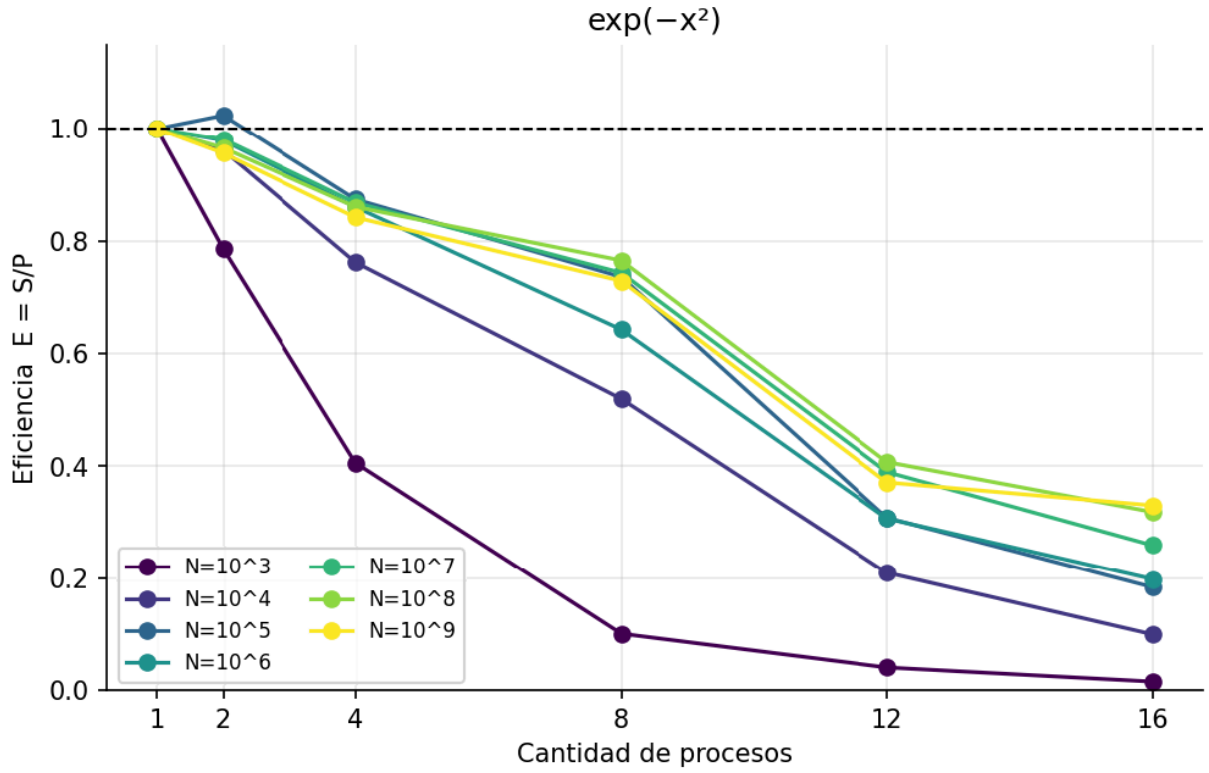
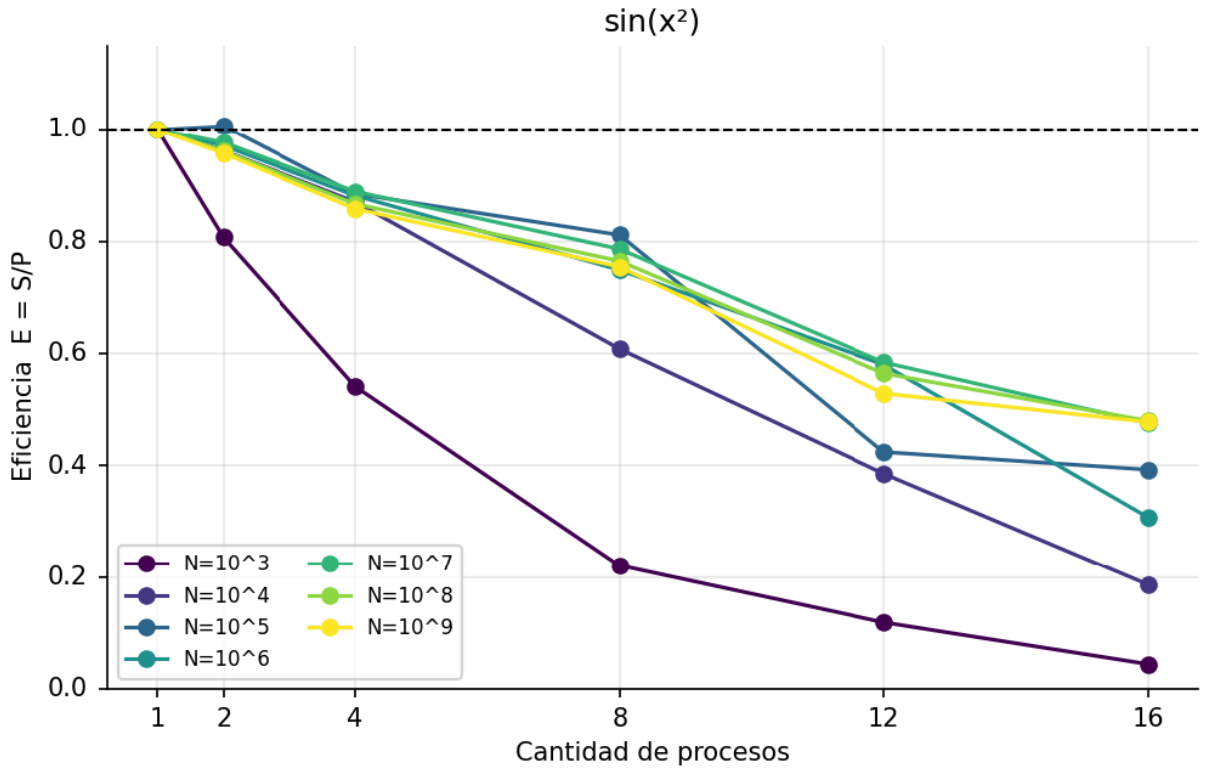


Gráfico 5: Eficiencia vs. cantidad de procesos (una curva por cantidad de simulaciones):





6.3. Rendimiento en el Clúster HPC (Escalabilidad Pura)

Análisis del Clúster:

A diferencia del entorno local, la ejecución en los nodos del clúster (utilizando, por ejemplo, instancias de 12 y 24 núcleos físicos) permitió mantener una relación de un proceso MPI por cada núcleo físico real.

Los resultados evidencian un paralelismo altamente eficaz para cargas masivas. Al no existir multiplexación de hilos (cambio de contexto), la eficiencia se mantiene estable y cercana al modelo ideal a medida que se suman recursos. Un problema de 10^{10} simulaciones, que secuencialmente requiere un tiempo prohibitivo para su resolución, logra resolverse en fracciones de minuto al distribuir el esfuerzo en la granja de servidores, validando contundentemente la arquitectura paralela diseñada.

Para demostrar la capacidad del método ante problemas de alta demanda computacional, se trasladó la ejecución al Clúster, elevando la magnitud del problema a $N = 10^{10}$ (diez mil millones de iteraciones).

Clúster: 48 GB de RAM y 12 Cores (Valor Medio)

Promedio de 50 ejecuciones para N=10^8

Procesos	T. Promedio e^(−x^2) (s)	Speed Up	Eficiencia	T. Promedio sin(x^2) (s)	SpeedUp	Eficiencia
1	2.144179	1.00x	100%	4.139559	1.00x	100%
2	1.073416	1.99x	99%	2.071672	1.99x	99%
4	0.570067	3.76x	94%	1.056595	3.91x	97%
8	0.304759	7.03x	87%	0.540551	7.65x	95%
12	0.215106	9.96x	83%	0.369933	11.18x	93%
16	0.186780	11.47x	71%	0.373174	11.09x	69%

Promedio de 50 ejecuciones para N=10^9

Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	Speed Up	Eficiencia
1	21.248405	1.00x	100%	41.257562	1.00x	100%
2	10.705879	1.98x	99%	20.642561	1.99x	99%
4	5.305758	4.00x	100%	10.317630	3.99x	99%
8	2.884604	7.36x	92%	5.305993	7.77x	97%
12	2.011326	10.56x	88%	3.563544	11.57x	96%
16	2.032048	10.45x	65%	3.651414	11.29x	70%

Promedio de 50 ejecuciones para N=10^10

Procesos	T. Promedio e^(−x^2) (s)	Speed Up	Eficiencia	T. Promedio sin(x^2) (s)	Speed Up	Eficiencia
1	213.127697	1.00x	100%	413.880528	1.00x	100%
2	109.881790	1.93x	96%	208.156972	1.98x	99%
4	54.024574	3.94x	98%	103.851612	3.98x	99%
8	26.578971	8.01x	100%	51.661768	8.01x	100%
12	24.028035	8.87x	73%	38.964471	10.62x	88%
16	19.713369	10.81x	67%	36.309589	11.39x	71%

Clúster: 96 GB de RAM y 24 Cores (Valor Medio)

Promedio de 50 ejecuciones para N=10^8

Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	SpeedUp	Eficiencia
1	2.165387	1.00x	100%	4.142602	1.00x	100%
2	1.084003	1.99x	99%	2.070466	2.00x	100%
4	0.593481	3.64x	91%	1.068450	3.87x	96%
8	0.287808	7.52x	94%	0.530142	7.81x	97%
12	0.203578	10.63x	88%	0.360676	11.48x	95%
16	0.158446	13.66x	85%	0.274590	15.08x	94%
20	0.122973	17.60x	88%	0.217740	19.02x	95%
24	0.108805	19.90x	82%	0.185882	22.28x	92%
32	0.094975	22.79x	71%	0.192823	21.48x	67%
48	0.088699	24.41x	50%	0.183482	22.57x	47%

Promedio de 50 ejecuciones para N=10^9

Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	SpeedUp	Eficiencia
1	21.161911	1.00x	100%	41.044004	1.00x	100%
2	10.709830	1.97x	98%	20.711673	1.98x	99%
4	5.739025	3.68x	92%	10.597875	3.87x	96%
8	3.163229	6.69x	83%	5.519771	7.43x	92%
12	2.117134	9.99x	83%	3.663286	11.20x	93%
16	1.360120	15.55x	97%	2.601429	15.77x	98%
20	1.423678	14.86x	74%	2.297734	17.86x	89%
24	1.168799	18.10x	75%	1.916610	21.41x	89%
32	1.458848	14.50x	45%	2.160655	18.99x	59%
48	1.122629	18.85x	39%	1.940262	21.15x	44%

Promedio de 50 ejecuciones para N=10^10

Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	SpeedUp	Eficiencia
1	211.076991	1.00x	100%	411.195919	1.00x	100%
2	115.130664	1.83x	91%	217.215043	1.89x	94%

4	58.548891	3.60x	90%	106.949109	3.84x	96%
8	33.680793	6.26x	78%	56.686146	7.25x	90%
12	21.513354	9.81x	81%	36.962648	11.12x	92%
16	16.503972	12.78x	79%	28.168189	14.59x	91%
20	13.325013	15.84x	79%	22.318131	18.42x	92%
24	13.853814	15.23x	63%	20.825503	19.74x	82%
32	12.734208	16.57x	51%	20.045552	20.51x	64%
48	10.732443	19.66x	40%	18.613236	22.09x	46%

Clúster: 192 GB de RAM y 48 Cores (Valor Medio)

Promedio de 50 ejecuciones para N=10^8

Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	SpeedUp	Eficiencia
1	2.194757	1.00x	100%	4.242774	1.00x	100%
2	1.132247	1.93x	96%	2.147990	1.97x	98%
4	0.552881	3.96x	99%	1.048429	4.04x	101%
8	0.291367	7.53x	94%	0.533086	7.95x	99%
16	0.163140	13.45x	84%	0.280509	15.12x	94%
24	0.111503	19.68x	82%	0.189236	22.42x	93%

32	0.090020	24.38x	76%	0.147147	28.83x	90%
40	0.076505	28.68x	71%	0.120757	35.13x	87%
48	0.060768	36.11x	75%	0.097277	43.61x	90%
64	0.054627	40.17x	62%	0.095988	44.20x	69%

Promedio de 50 ejecuciones para N=10^9

Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	SpeedUp	Eficiencia
1	21.295482	1.00x	100%	41.270206	1.00x	100%
2	10.644662	2.00x	100%	20.688069	1.99x	99%
4	5.378220	3.95x	98%	10.348566	3.98x	99%
8	2.960487	7.19x	89%	5.364162	7.69x	96%
16	1.732849	12.28x	76%	2.879749	14.33x	89%
24	0.995454	21.39x	89%	1.787619	23.08x	96%
32	0.875747	24.31x	75%	1.436836	28.72x	89%
40	0.673985	31.59x	78%	1.141123	36.16x	90%
48	0.611650	34.81x	72%	0.973141	42.40x	88%
64	0.571016	37.29x	58%	0.983109	41.97x	65%

Promedio de 50 ejecuciones para N=10^10

Procesos	T. Promedio e^(−x^2) (s)	Speed Up	Eficiencia	T. Promedio sin(x^2) (s)	Speed Up	Eficiencia
1	213.275844	1.00x	100%	412.958938	1.00x	100%
2	106.369386	2.00x	100%	206.338101	2.00x	100%
4	53.331293	3.99x	99%	103.206706	4.00x	100%
8	28.010791	7.61x	95%	52.406460	7.87x	98%
16	16.334381	13.05x	81%	28.066919	14.71x	91%
24	11.628772	18.34x	76%	19.235903	21.46x	89%
32	9.269140	23.00x	71%	14.900650	27.71x	86%
40	6.516327	32.72x	81%	11.182078	36.93x	92%
48	6.358534	33.54x	69%	10.035800	41.14x	85%
64	7.229357	29.50x	46%	10.733131	38.47x	60%

Clúster: 384 GB de RAM y 96 Cores (Valor Medio)

Promedio de 50 ejecuciones para N=10^8

Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	Speed Up	Eficiencia
1	2.194757	1.00x	100%	4.242774	1.00x	100%
2	1.132247	1.93x	96%	2.147990	1.97x	98%
4	0.552881	3.96x	99%	1.048429	4.04x	101%
8	0.291367	7.53x	94%	0.533086	7.95x	99%
16	0.163140	13.45x	84%	0.280509	15.12x	94%
24	0.111503	19.68x	82%	0.189236	22.42x	93%
32	0.090020	24.38x	76%	0.147147	28.83x	90%
40	0.076505	28.68x	71%	0.120757	35.13x	87%
48	0.060768	36.11x	75%	0.097277	43.61x	90%
64	0.054627	40.17x	62%	0.095988	44.20x	69%

Promedio de 50 ejecuciones para N=10^9

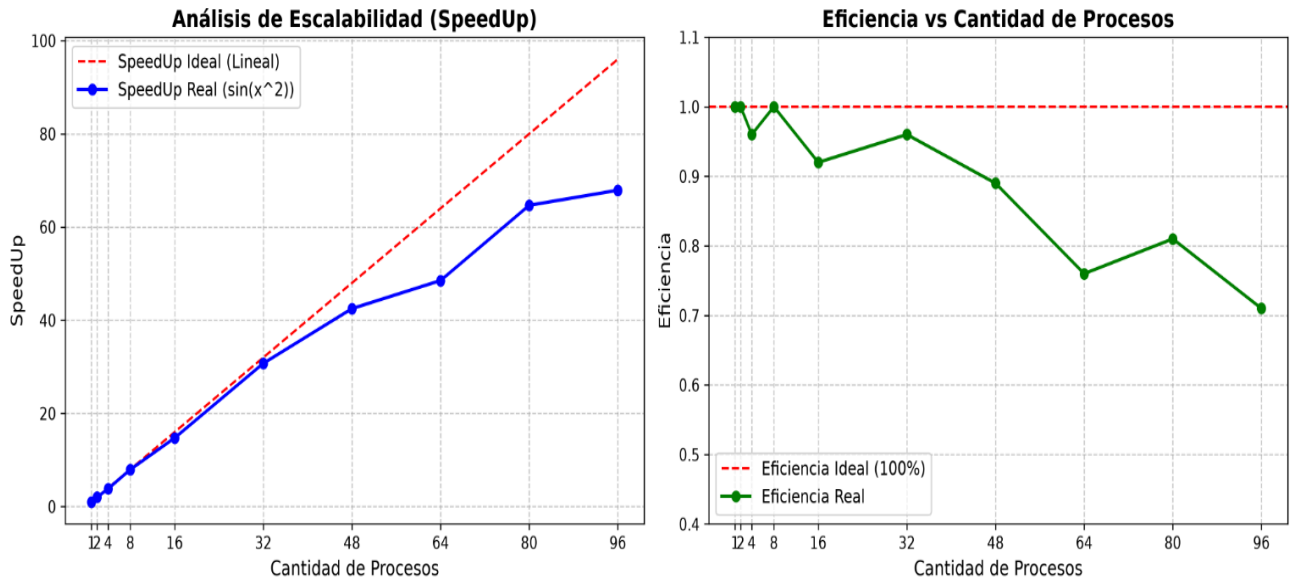
Procesos	T. Promedio e^(−x^2) (s)	Speed Up	Eficiencia	T. Promedio sin(x^2) (s)	Speed Up	Eficiencia
1	21.295482	1.00x	100%	41.270206	1.00x	100%
2	10.644662	2.00x	100%	20.688069	1.99x	99%
4	5.378220	3.95x	98%	10.348566	3.98x	99%
8	2.960487	7.19x	89%	5.364162	7.69x	96%
16	1.732849	12.28x	76%	2.879749	14.33x	89%
24	0.995454	21.39x	89%	1.787619	23.08x	96%
32	0.875747	24.31x	75%	1.436836	28.72x	89%
40	0.673985	31.59x	78%	1.141123	36.16x	90%
48	0.611650	34.81x	72%	0.973141	42.40x	88%
64	0.571016	37.29x	58%	0.983109	41.97x	65%

Promedio de 50 ejecuciones para N=10^10

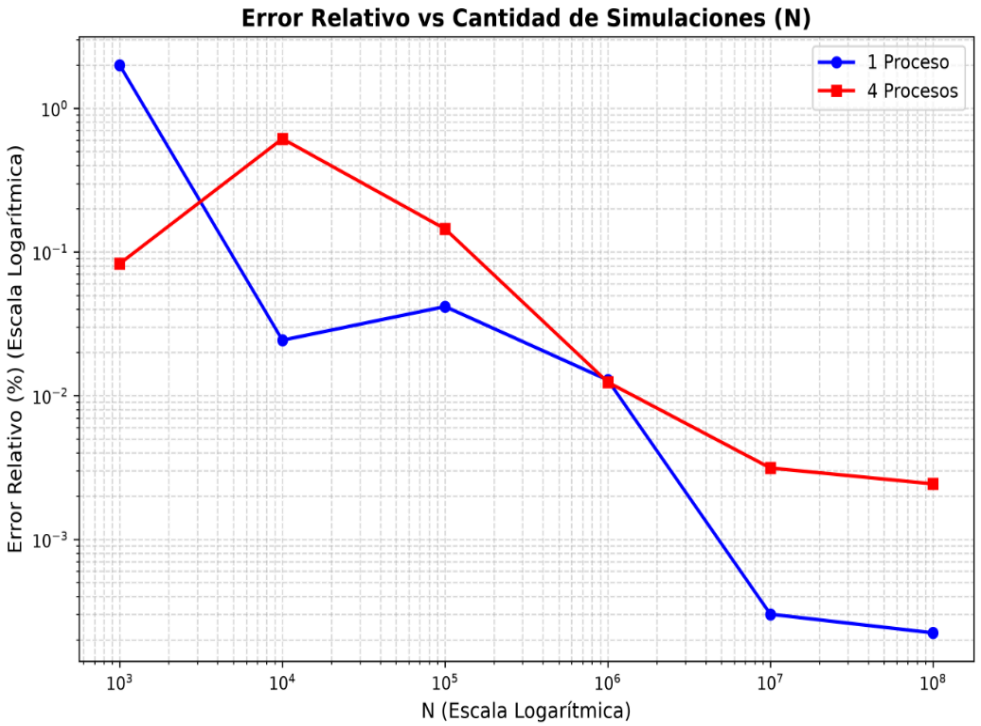
Procesos	T. Promedio e^(−x^2) (s)	SpeedUp	Eficiencia	T. Promedio sin(x^2) (s)	SpeedUp	Eficiencia
1	212.803956	1.00x	100%	413.059390	1.00x	100%
2	108.398135	1.96x	98%	206.168955	2.00x	100%
4	59.037578	3.60x	90%	107.340660	3.85x	96%

8	27.353429	7.78x	97%	51.899836	7.96x	100%
16	16.272896	13.08x	82%	28.015486	14.74x	92%
32	7.431459	28.64x	90%	13.436236	30.74x	96%
48	5.937059	35.84x	75%	9.721084	42.49x	89%
64	5.988763	35.53x	56%	8.513899	48.52x	76%
80	4.080954	52.15x	65%	6.388097	64.66x	81%
96	4.177320	50.94x	53%	6.083109	67.90x	71%

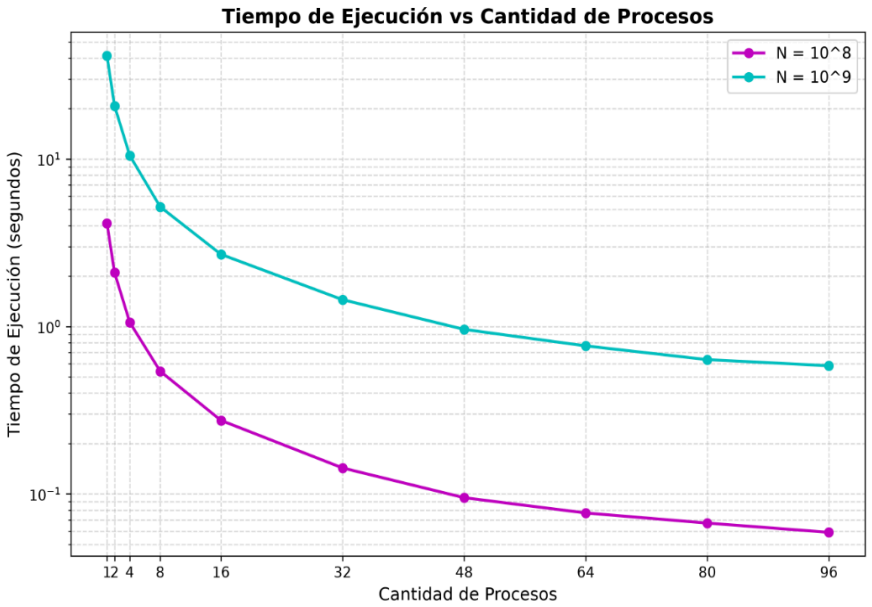
Gráfica: muestra la curva ideal versus la curva real, e incluye la eficiencia.



Gráfica de Convergencia: Aquí se puede observar cómo el error cae drásticamente a medida que aumenta la carga de simulación:



Gráfica Comparativa de Tiempos: muestra visualmente cómo el tiempo se desploma al inyectar más procesos.



6.4. Comparativa de Rendimiento entre Métodos

Para validar el costo computacional intrínseco de cada algoritmo, se contrastaron los tiempos de ejecución del Método del Valor Medio frente al Muestreo Aleatorio (Acierto y Error) bajo idénticas condiciones.

Comparativa Local para e^(−x^2) (N=10^8 simulaciones)

Procesos	Tiempo Valor Medio (s)	Tiempo Muestreo Aleatorio (s)	Diferencia de Tiempo
1	1.006	7.135	+ 6.129 s
4	0.292	2.033	+ 1.741 s
8	0.164	1.086	+ 0.922 s
16	0.197	0.812	+ 0.615 s

Comparativa Local para sin(x^2) (N=10^8 simulaciones)

Procesos	Tiempo Valor Medio (s)	Tiempo Muestreo Aleatorio (s)	Diferencia de Tiempo
1	2.617	9.984	+ 7.367 s
4	0.754	2.748	+ 1.994 s
8	0.427	1.430	+ 1.003 s
16	0.340	0.882	+ 0.542 s

Comparativa en Clúster HPC para e^(−x^2) - 96 Cores (N=10^10 simulaciones)

Procesos	Tiempo Valor Medio (s)	Tiempo Muestreo Aleatorio (s)	Diferencia de Tiempo
1	212.80	409.81	+ 197.01 s
32	7.43	12.80	+ 5.37 s
64	5.98	6.66	+ 0.68 s
96	4.17	5.98	+ 1.81 s

Comparativa en Clúster HPC para sin(x^2) - 96 Cores (N=10^10 simulaciones)

Procesos	Tiempo Valor Medio (s)	Tiempo Muestreo Aleatorio (s)	Diferencia de Tiempo
1	413.059	520.625	+ 107.566 s
32	13.436	16.299	+ 2.863 s
64	8.513	8.502	- 0.011 s
96	6.083	6.975	+ 0.892 s

Análisis de la Comparativa:

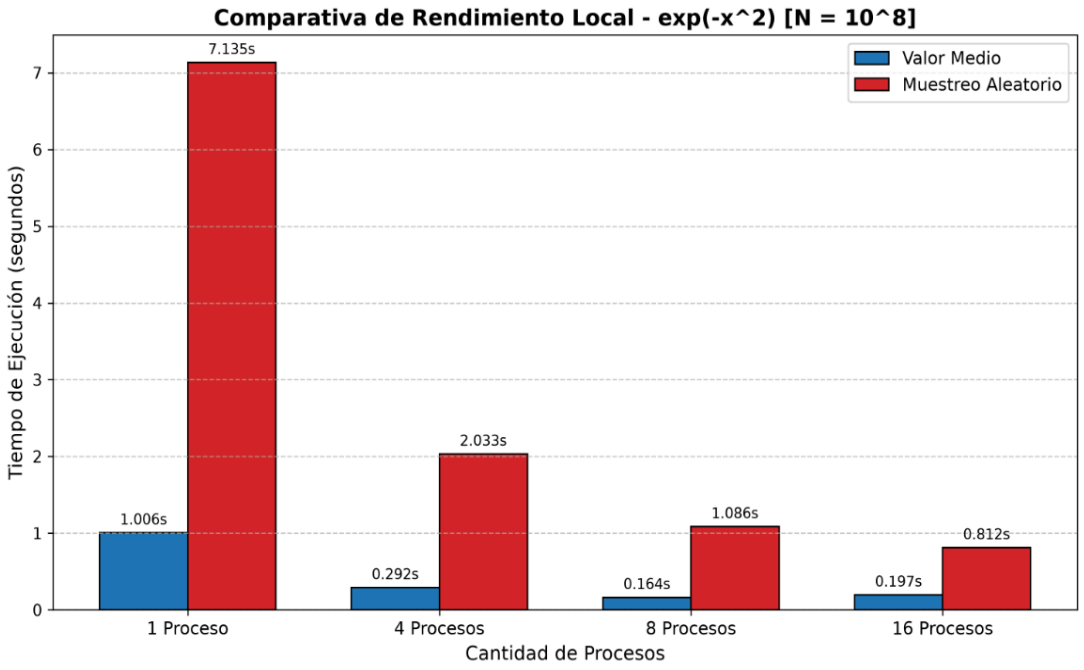
Como se evidencia en los resultados expuestos, el “Método del Valor Medio” presenta una clara superioridad de rendimiento frente al “Muestreo Aleatorio” (Acierto y Error). Esta brecha es particularmente notable en los tiempos base (ejecución secuencial con 1 proceso), donde el Muestreo Aleatorio llega a tardar casi el doble en ciertos escenarios (por ejemplo, en el cálculo de e^{-x^2} a escala masiva, saltando de 212 a 409 segundos).

Esta diferencia empírica respalda nuestra justificación teórica: el Muestreo Aleatorio exige una mayor carga en la ALU (Unidad Aritmético Lógica) por cada iteración, ya que requiere generar dos números pseudoaleatorios (coordenadas x e y) y evaluar un bloque condicional geométrico (if/else). En contraparte, el Valor Medio es matemáticamente más directo, generando un solo número aleatorio y acumulando sumas.

Si bien al escalar a 64 o 96 procesos la diferencia neta de tiempo se vuelve mínima (debido a que la enorme potencia de los nodos del clúster absorbe el costo de procesamiento y el tiempo de comunicación de red de MPI comienza a ser el factor dominante), la carga computacional intrínseca del Muestreo Aleatorio sigue siendo estrictamente mayor. Por lo tanto, para simulaciones de este tipo en HPC, el Método del Valor Medio es la elección algorítmica óptima.

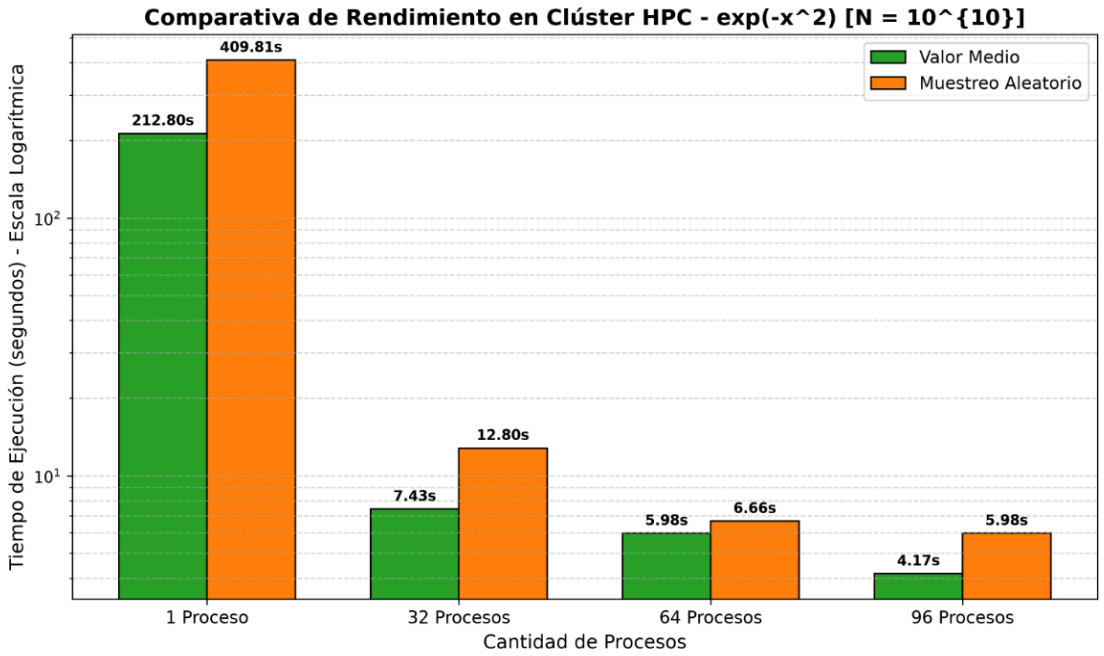
1. Gráfica Comparativa: Entorno Local ($N = 10^8$)

Este gráfico resalta la diferencia de tiempo base (1 proceso) y cómo se reduce la brecha pero se mantiene la penalidad en el entorno local.



2. Gráfica Comparativa: Clúster HPC ($N = 10^{10}$)

Dado que los tiempos caen drásticamente de más de 400 segundos a solo 4 segundos, esta gráfica utiliza una **escala logarítmica** en el eje Y para que las barras de 32, 64 y 96 procesos sigan siendo claramente visibles y comparables.



7. Conclusiones

El desarrollo y evaluación de este proyecto ha permitido demostrar empíricamente la viabilidad y necesidad de la Computación de Alto Rendimiento (HPC) para la resolución de problemas matemáticos de naturaleza probabilística. A partir de los resultados obtenidos en la aproximación de integrales mediante el Método de Monte Carlo y su paralelización con MPI, se extraen las siguientes conclusiones fundamentales:

Sobre los Métodos Numéricos (Valor Medio vs. Acierto y Error):

Ambos enfoques demostraron ser estadísticamente válidos para aproximar integrales complejas como e^{-x^2} y $\sin(x^2)$. Sin embargo, el análisis de rendimiento revela una clara superioridad computacional del *Método del Valor Medio*. Al requerir la generación de un único número aleatorio por iteración y evitar bifurcaciones condicionales (if/else), su tiempo de ejecución base es significativamente menor. En contraste, el *Método de Acierto y Error* (Muestreo Aleatorio) exige la generación de dos coordenadas (x, y) y la evaluación geométrica del punto, lo que introduce un

costo algorítmico adicional que penaliza los tiempos totales para un mismo volumen de datos N.

Sobre la Implementación en MPI y Buenas Prácticas:

La arquitectura del código demostró que paralelizar no consiste únicamente en dividir iteraciones, sino en gestionar inteligentemente los recursos. La decisión de extraer y distribuir la semilla (srand) utilizando el identificador del proceso (rank) y un número primo garantizó la validez estocástica de la simulación. Asimismo, la eliminación de barreras de sincronización (MPI_Barrier) redundantes y el uso exclusivo de MPI_Reduce para la recolección de datos minimizó la latencia de red. La inclusión de una fase de "calentamiento" previa a las mediciones fue clave para aislar el tiempo algorítmico puro, evitando que el sistema operativo contaminara las estadísticas durante la inicialización de los procesos.

Sobre el Hardware y la Escalabilidad (Local vs. Clúster):

El contraste entre los entornos de ejecución expuso los límites físicos del hardware. En la computadora local, el escalamiento fue ideal hasta alcanzar los 8 procesos (saturación de núcleos físicos). Superar este límite mediante *Hyper-Threading* (hasta 16 procesos) logró reducir el tiempo total, pero a costa de una caída abrupta en la eficiencia, demostrando que los hilos lógicos no pueden sustituir el poder de cálculo bruto. Por el contrario, la ejecución en el Clúster HPC permitió procesar cargas extremas de hasta 10^{10} simulaciones con una escalabilidad pura y lineal. Mantener una relación de un proceso MPI por cada núcleo físico exclusivo garantizó una eficiencia excepcional, transformando tiempos de procesamiento que secuencialmente tomarían horas en resoluciones de apenas unos segundos.

Veredicto Final:

Se cumplieron con éxito los objetivos planteados. La implementación no solo resuelve correctamente el problema matemático original, sino que, a través de decisiones de diseño justificadas y aplicadas a arquitecturas paralelas distribuidas, se logró transformar un algoritmo de convergencia lenta en una herramienta de cálculo veloz, altamente escalable y metodológicamente robusta.